凱撒密碼破解器

Caesar Cipher Analyzer | 以 CustomTkinter 打造的現代化密碼分析工具

Python 3.14 · CustomTkinter 5.2.2 · PyInstaller 6.22.3 · Windows 桌面應用 · 支援深色模式與單一 exe 打包

GitHub 專案倉庫 https://github.com/Hilton-svg/SBA-work

本專案是一套完整的**凱撒密碼破解工具**：運算核心在 **Untitled-1.py**（字母統計、卡方檢定、n-gram 命中率、元音比例與凱撒解密），互動介面在 **caesar_gui.py**（CustomTkinter 桌面視窗），兩者以函式呼叫串接。程式對 26 種可能的位移逐一評分，並把輸出編排為**左右兩個區塊**——右方放「最有可能偏移量與該偏移量下的解密結果」，左方放其餘所有統計數據。

01 專案特色

特色	說明
雙區塊輸出	右方區塊 = 主要結果（最有可能偏移量 k、混合分數、該 k 的完整解密全文、候選前三名切換）；左方區塊 = 其餘數據（字母頻率直方圖、26 種位移分數明細、候選排名、字母總數與動態權重）。
深色模式	提供「跟隨系統 / 淺色 / 深色」即時切換；所有顏色以 (淺色, 深色) 色票定義，切換不需重建介面。
現代化版面	圓角卡片式配置、彩色進度條直方圖、等寬字型數據表、狀態列即時顯示耗時與結論。
固定最大化視窗	啟動即以 zoomed 狀態填滿工作區並鎖定尺寸（resizable 關閉），版面永遠一致、不會被誤縮放。
一鍵打包 exe	build_exe.bat 可產出單一檔案 exe（13.1 MB）或免解壓資料夾版；核心程式與範例檔一併內嵌。
可重現的自我測試	內建 --selftest：12 項功能檢查（分析、左右區塊內容、候選切換、剪貼簿、清空、無字母輸入等），以結束碼 0/1 表示通過與否，適合自動化驗證。

02 快速開始

三種啟動方式，任選一種即可：

```
1) 雙擊執行檔（推薦，免安裝）
   dist\CaesarCipherGUI.exe # 單一檔案版
   dist_portable\CaesarCipherGUI\CaesarCipherGUI.exe # 資料夾版，啟動最快

2) 雙擊批次檔：run_gui.bat（自動使用專案 .venv）

3) 命令列：.venv\Scripts\python.exe caesar_gui.py [--demo | --selftest]
```

> **操作提示**
> 在輸入框貼上密文後按 **開始分析**（或直接按 **Ctrl + Enter**）即可破解。也可以按「從檔案載入」讀取 .txt，或按「載入範例密文」載入 test.txt 中自動偵測到的密文段落。

03 介面導覽

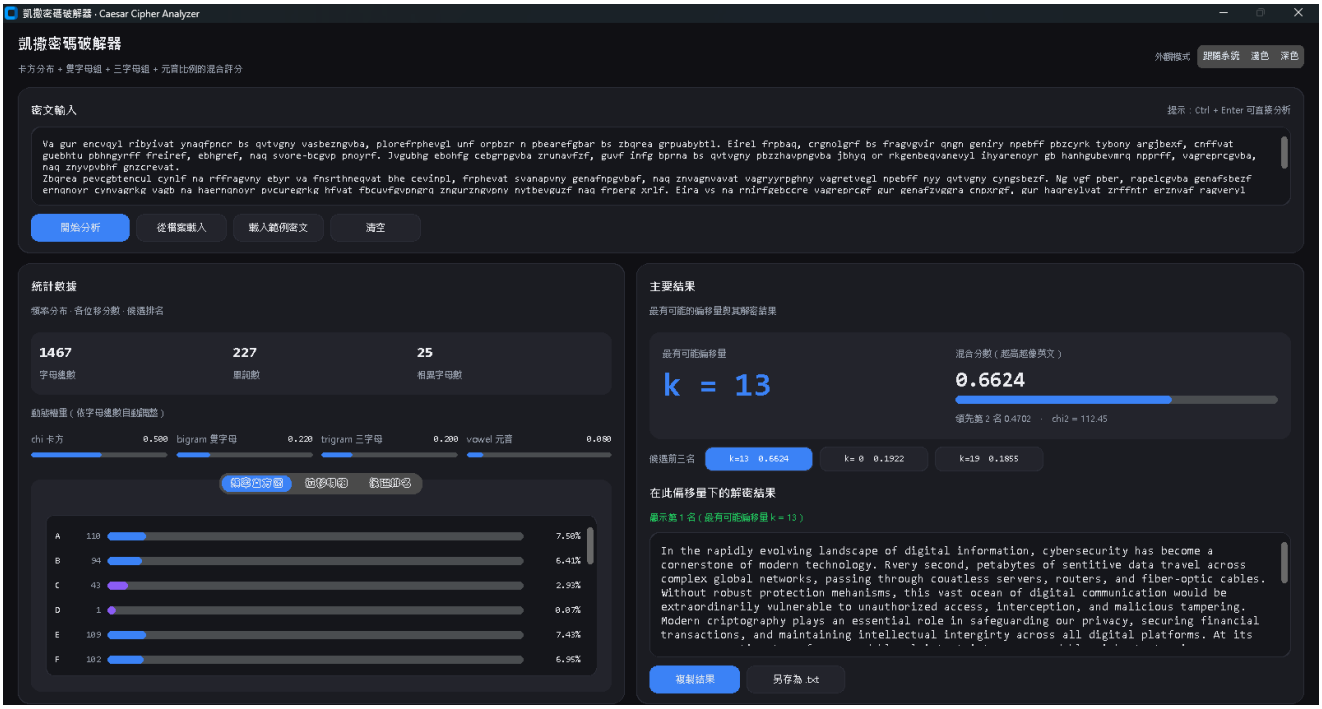


圖 1 深色模式（固定最大化）：左方為統計數據，右方為主要結果

區域	內容
標題列	程式名稱與「外觀模式」三段切換鈕（跟隨系統 / 淺色 / 深色）。
輸入區	密文輸入框，以及 **開始分析**、**從檔案載入**、**載入範例密文**、**清空** 四個按鈕。
左方：統計數據	字母總數 / 單詞數 / 相異字母數、四項動態權重長條，以及三個分頁：**頻率直方圖**（A–Z 彩色長條，可切換顯示 0 次字母）、**位移明細**（k = 0–25 的 chi2 / chi / bigram / trigram / vowel / mixed 與預覽，最佳者標 ★）、**候選排名**（依混合分數排序的完整表格）。
右方：主要結果	大字顯示**最有可能偏移量 k** 與混合分數（含進度條、領先差距、chi2），候選前三名可點選切換，下方為**該偏移量下的完整解密全文**，並提供複製與另存 .txt。
狀態列	顯示分析結果摘要：位移數、字母總數、最可能 k、混合分數與耗時（毫秒）。

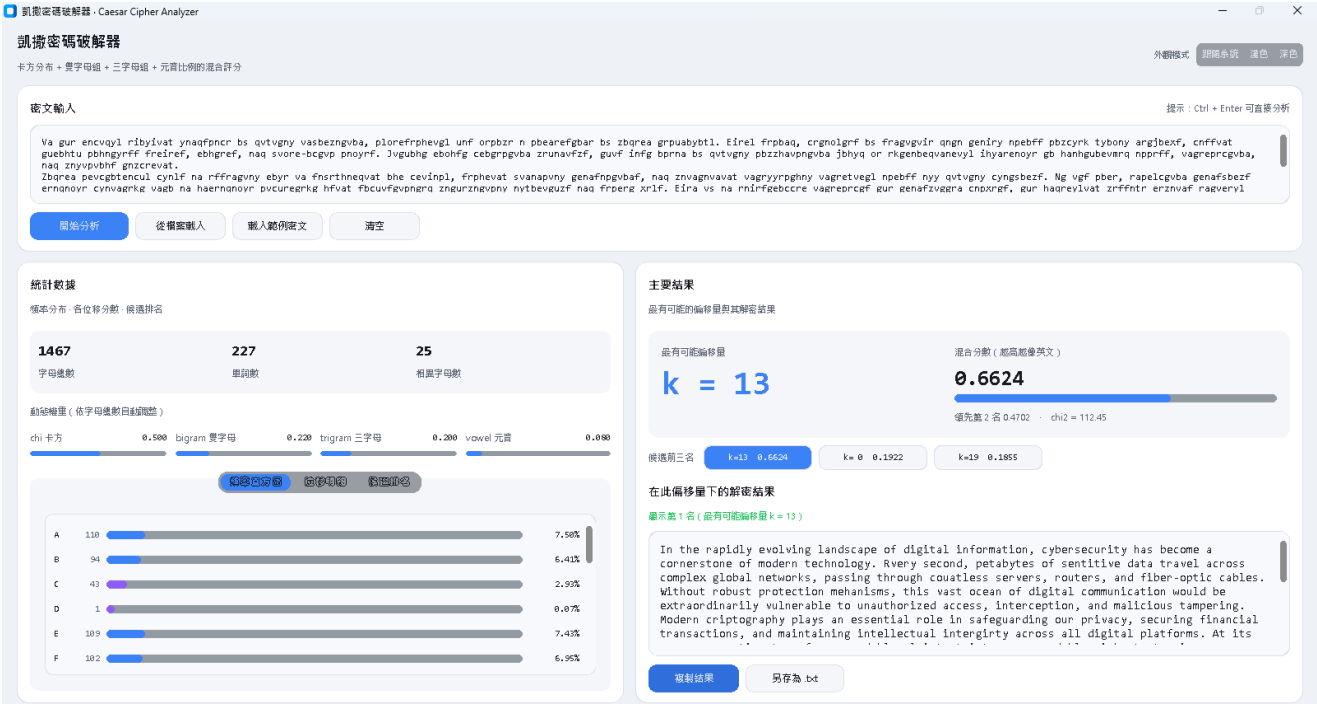


圖 2 淺色模式：同一份版面，僅外觀色票不同

04 操作流程

**步驟 1	輸入密文**	在輸入區貼上要破解的英文字母密文（標點、數字、空白會保留但不列入評分）。
**步驟 2	開始分析**	按「開始分析」或 Ctrl + Enter。程式會統計字母分布、取得動態權重，再對 26 種位移逐一解密與評分。
**步驟 3	看右方結論**	右方大字顯示最有可能的偏移量 k 以及混合分數；下方文字框即為該 k 的解密全文。
**步驟 4	驗證其他候選**	若結果讀不通，可點選右方的候選第 2、3 名按鈕，立即切換顯示該位移的全文；狀態列會提示目前顯示的是第幾名。
**步驟 5	取用結果**	按「複製結果」複製到剪貼簿，或「另存為 .txt」存成 decrypted_k<偏移量>.txt。

05 核心程式：Untitled-1.py（程式主體）

本專案的運算核心是 **Untitled-1.py**（410 行、15.4 KB）：所有密碼學運算都在這裡完成——字母統計、卡方檢定、n-gram 命中率、元音比例、凱撒解密與最佳位移排序。**caesar_gui.py** 負責操作介面：它載入核心模組，呼叫其中的函式取得結果並繪製成畫面。兩者職責分明——核心可獨立以命令列執行，也能被介面重複呼叫。完整原始碼收錄於**附錄 B**。

> **核心與介面的銜接方式**
> 執行 `caesar_gui.py` 時，它會載入同資料夾的 `Untitled-1.py` 作為運算模組。因為檔名含有「-」，無法使用一般 `import` 語法，所以改用 `importlib.util.spec_from_file_location("caesar_core", path)` 動態載入；模組名稱設為 `caesar_core`，核心檔案最後的 `if __name__ == "__main__"` 因此不會成立，它的命令列流程不會被介面意外觸發。
> 打包成 exe 後，程式會**優先讀取 exe 旁邊的同名 Untitled-1.py**，找不到才使用內嵌版本；因此替換這一個檔案就能更換運算核心，不必重新打包。

模組層級常數與資料表

名稱	內容	用途
ENGLISH_FREQ	26 個字母的英文出現頻率（%，總和 100）	卡方檢定的期望分布。
COMMON_BIGRAMS	39 組常見雙字母組（th、he、in…）	計算 bigram 命中率。
COMMON_TRIGRAMS	28 組常見三字母組（the、and、ing…）	計算 trigram 命中率。
CHI_WEIGHT 等四項	0.35 / 0.28 / 0.27 / 0.10	呼叫端未指定權重時的預設值。
TARGET_VOWEL_RATIO	0.38	一般英文文本的元音比例目標。

函式一覽（共 14 個，依檔案順序）

函式	回傳	說明
get_dynamic_weights	4-tuple	依字母總數在「短文本權重」與「長文本權重」之間線性插值。
count_letters	dict	統計 A–Z 每個字母的出現次數，不分大小寫；未出現的字母為 0。
chi_square_score	float	卡方統計量 Σ(觀測 - 期望)² / 期望；完全沒有字母時回傳無限大。
extract_words	list	以非英文字母為邊界切出所有單詞，標點與數字不會被計入。
bigram_score	float	單詞內相鄰兩字母命中 COMMON_BIGRAMS 的比例，不跨越標點。
trigram_score	float	同上但取相鄰三字母，比 bigram 更能反映真實英文結構。
vowel_score	float	元音佔比與 0.38 的接近程度，回傳 0～1。
normalize_chi_score	float	1 / (1 + chi2 / 字母數)，把「越低越好」的卡方轉成 0～1 的分數。
compute_all_scores	6-tuple	一次算好卡方與四項分數及字母總數，避免重複計算。
mixed_score_from_parts	float	以既有分數與權重加權合成混合分數（四項權重和為 1）。
mixed_score	float	混合評分；未指定權重時自動採用動態權重；無字母回傳 -1.0。
print_histogram	None	命令列版的文字直方圖（1 符號 = 1%）；GUI 改以彩色進度條重繪。
decryption	str	凱撒解密：字母往前移 k 位，大小寫保持，其他字元原樣保留。
main	None	命令列流程：讀入文字 → 直方圖 → 26 種位移評分 → 印出最佳解。

GUI 實際呼叫其中 7 個函式：count_letters、extract_words、get_dynamic_weights、decryption、compute_all_scores、mixed_score_from_parts 與 mixed_score。其餘的 chi_square_score、bigram_score、trigram_score、vowel_score、normalize_chi_score 由 compute_all_scores 在內部呼叫；print_histogram 與 main 屬於命令列介面，GUI 以自己的版面取代。

06 運作原理

① 輸入密文	② count_letters	③ get_dynamic_weights	**④ 26 次 decryption**
⑤ compute_all_scores	⑥ mixed_score_from_parts	**⑦ 排序取最佳 k**	**⑧ 左方數據 / 右方結果**

流程由左至右、上至下：先統計 → 決定權重 → 對每個位移解密並計算四項分數 → 加權合成 → 排序顯示。

混合評分（分數越高越像英文）

分項	計算方式	方向
chi 卡方	對解密結果做卡方檢定：Σ (觀測次數 - 期望次數)² / 期望次數，期望值取自一般英文單字母頻率；再正規化為 1 / (1 + chi2 / 字母數)。	越低越好 → 正規化後越高越好
bigram 雙字母	以非字母字元切出單詞後，統計單詞內相鄰兩字母命中常見雙字母組（th、he、in…）的比例，不跨標點。	越高越好
trigram 三字母	同上但取相鄰三字母（the、and、ing…），比 bigram 更能反映真實英文結構。	越高越好
vowel 元音	元音（a/e/i/o/u）佔全部字母的比例，與目標值 0.38 的距離越近分數越高：1 - |比例 - 0.38| / 0.38。	越接近 0.38 越好

動態權重：依文本長度自動調整四項比重

文本長度（字母數）	chi	bigram	trigram	vowel
<= 20（短文本，卡方不穩定）	0.15	0.35	0.25	0.25
20 ~ 200（線性插值過渡）	0.15→0.50	0.35→0.22	0.25→0.20	0.25→0.08
>= 200（長文本，卡方穩定）	0.50	0.22	0.20	0.08

插值：t = 0（n <= 20）| t = (n - 20) / 180（20 < n < 200）| t = 1（n >= 200）
混合分數 = w_chi * chi + w_bigram * bigram + w_trigram * trigram + w_vowel * vowel
本範例（test.txt，1467 字母）取得長文本權重：0.500 / 0.220 / 0.200 / 0.080

07 檔案結構

檔案 / 資料夾	行數 / 大小	說明
caesar_gui.py	1241 行 / 49 KB	互動介面：版面組裝、事件處理、分析流程、自我測試與打包相容處理。
Untitled-1.py	410 行 / 15.4 KB	**運算核心**：字母統計、卡方、bigram / trigram、元音與凱撒解密，並附命令列版本；介面直接呼叫其中的函式。完整原始碼見附錄 B。
test.txt	9 行 / 3.5 KB	範例資料：前 4 段明文 + 後 4 段 ROT13 密文；程式會自動判斷並只取密文段落。
run_gui.bat	10 行	以專案 .venv 啟動圖形介面（雙擊即可）。
build_exe.bat	25 行	一鍵打包 exe，內部呼叫 .tools\build_exe.py。
.tools\	5 個檔案	打包與環境工具：build_exe.py（PyInstaller 設定）、pip_patch.py（沙箱可用的 pip 包裝）、make_intro_pdf.py（本介紹文件的產生腳本）、兩張介面截圖。
dist\CaesarCipherGUI.exe	13.1 MB	單一檔案執行檔（onefile），雙擊即用。

檔案 / 資料夾	行數 / 大小	說明
`dist_portable\`	`29.5 MB`	免解壓資料夾版（onedir，主程式 3.2 MB），啟動最快。
`CaesarCipherGUI.spec`	—	PyInstaller 打包設定檔，可自行調整後重打包。
`.venv\`	—	虛擬環境：customtkinter 5.2.2、darkdetect 0.8.0、pyinstaller 6.22.3 等。

專案倉庫（GitHub）
`https://github.com/Hilton-svg/SBA-work`，公開倉庫，內容與本文件所述一致，可直接以 `git clone` 取得完整專案（含程式主體 Untitled-1.py）。

08 打包成執行檔（exe）

雙擊 **build_exe.bat** 即可重新打包；或執行 `python .tools\build_exe.py [--onedir]`。打包指令使用 PyInstaller 的 onefile / windowed 模式，並以 `--collect-all customtkinter` 一併收錄主題 JSON 與字型資源。

版本	產物	體積	特性
單一檔案（onefile）	`dist\CaesarCipherGUI.exe`	`13.1 MB`	只有一個檔案、方便攜帶；啟動時會先解壓到 %TEMP% 再執行，首次啟動約 2–4 秒。
資料夾版（onedir）	`dist_portable\CaesarCipherGUI\`	`29.5 MB`	不需解壓、啟動最快，適合放在固定位置長期使用。

兩項打包設計
① **外部檔案優先**：資料檔（Untitled-1.py、test.txt）會先找 exe 旁邊的同名檔案，找不到才用內嵌版本；因此只要把修改後的 Untitled-1.py 放到 exe 同資料夾，就能換掉演算法而不必重新打包。
② **內建診斷記錄**：打包後的程式會把 --selftest 結果與未預期例外寫入 `%TEMP%\caesar_gui.log`（視窗模式沒有主控台可看），方便回報問題。

09 驗證與實測結果

程式內建 12 項自我測試（`--selftest`），原始碼與打包後的 exe 皆為 **12/12 通過、結束碼 0**：

檢查項目	檢查內容
分析產生 26 種位移	對 k = 0–25 全部完成解密與評分。
找出最有可能偏移量	最佳 k 不為 None，且混合分數取最大值。
範例密文最佳位移為 13	test.txt 的密文為 ROT13，程式須正確判為 k = 13。
左右區塊內容	位移明細與候選排名文字長度正常、右方顯示出正確明文關鍵字。
直方圖有資料列	頻率直方圖確實產生 25 列（非零字母數）。
候選切換、複製、清空	切換到第 2 名、剪貼簿內容與結果框一致、清空後回復初始狀態。
邊界情境	純數字 / 符號輸入（無英文字母）不會崩潰，並顯示「找不到任何英文字母」。
載入範例	「載入範例密文」能正確讀入並填入輸入框。

以 test.txt 的密文段切實測（1467 個字母）

項目	結果	項目	結果
字母總數	1467	單詞數	227
相異字母數	25	嘗試位移數	26
最可能偏移量	k = 13（ROT13）	混合分數	0.6624
領先第 2 名	0.4702	chi2	112.45
分析耗時	約 0.44–0.53 秒（含左方重繪）	自我測試	12/12 通過（exit 0）

10 疑難排解

Q：找不到核心程式檔案：Untitled-1.py

A：GUI 與 Untitled-1.py 必須放在同一個資料夾（原始碼模式），或把 Untitled-1.py 放到 exe 旁邊。

Q：第一次啟動 exe 要等幾秒

A：單一檔案版啟動時會把內容解壓到 %TEMP%_MEIxxxxxx，屬於正常現象；若想秒開請改用資料夾版。

Q：結果讀起來不通順

A：密文太短（少於 20 個字母）時卡方統計不穩定，請多貼一些內容；也可直接點選右方候選第 2、3 名比較。

Q：想確認程式內部狀態

A：執行 --selftest 會產生 12 項檢查；打包後的版本請看 %TEMP%\caesar_gui.log。

Q：防毒軟體攔阻 exe

A：PyInstaller 產生的未簽章執行檔常被誤判，可改用資料夾版或將檔案加入白名單。

Q：想調整演算法或介面

A：演算法改 Untitled-1.py，畫面改 caesar_gui.py，兩者以函式名稱溝通；若只想換掉運算核心，直接替換 exe 旁邊的 Untitled-1.py 即可，不必重新打包。

11 附錄 A：命令列參數與環境

參數	說明
`--demo`	啟動後自動載入 test.txt 的密文段落並立即分析。
`--selftest`	執行 12 項自我測試後自動關閉；結束碼 0 表示全部通過。
`--appearance=…`	指定啟動外觀：深色 / 淺色 / 跟隨系統（亦可寫 Dark / Light / System）。
`--tab=…`	啟動後直接切到左方指定分頁：頻率直方圖 / 位移明細 / 候選排名。
`--auto-close=秒`	指定秒數後自動關閉視窗（供截圖與自動化使用）。
`Ctrl + Enter`	在輸入框中直接開始分析。

元件	版本 / 內容	用途
`Python`	3.14.6（.venv 虛擬環境）	執行 GUI 與打包。
`CustomTkinter`	5.2.2	現代化深色模式介面元件。
`darkdetect`	0.8.0	「跟隨系統」外觀偵測。
`PyInstaller`	6.22.3	打包成單一 exe / 資料夾版。
字型	Microsoft JhengHei UI / Consolas	中文介面文字與等寬數據表格。

本文件由專案內 .tools 的產生腳本自動排版輸出（reportlab + Microsoft JhengHei）。文件中的所有數據均取自實際執行的測試結果。

12 附錄 B：Untitled-1.py 完整原始碼

以下是程式主體 **Untitled-1.py** 的完整內容：410 行、15.4 KB、UTF-8 編碼，只使用 Python 標準函式庫（`re`、`string`、`collections`）。本清單與 GitHub 倉庫中的檔案一致（https://github.com/Hilton-svg/SBA-work）。

```
# -*- coding: utf-8 -*-
"""
Untitled-1.py：凱撒密碼（Caesar cipher）破解 + 英文相似度評分工具

功能概述：
    1. 讀入一段文字（通常是密文）。
    2. 統計其中 A-Z 出現次數，並用文字直方圖顯示字母頻率。
    3. 依文本長度決定四項評分的「動態權重」。
    4. 對 26 種可能的位移 k 逐一凱撒解密，計算每個結果的混合分數。
    5. 印出所有 k 的分數明細，並輸出分數最高（最像英文）的解密結果。

混合評分（分數越高越像英文）由四項加權而成：
    chi      ：卡方分數正規化（字母頻率分布是否接近一般英文）
    bigram   ：常見雙字母組（bigram）命中率
    trigram  ：常見三字母組（trigram）命中率
    vowel    ：元音比例是否接近 TARGET_VOWEL_RATIO
"""

import re                           # 用於以正規表示式切出單詞
import string                       # 提供 ascii_uppercase 等常數
from collections import Counter     # 用於快速統計字元出現次數

# ==================== 常數表 ====================

# 英文單字母頻率（%）
# 一般英文書面文本中各字母出現的百分比，作為卡方分數的「期望分布」。
# 鍵為 A-Z 大寫字母，數值為百分比（總和約 100）。
ENGLISH_FREQ = {
    'A': 8.167, 'B': 1.492, 'C': 2.782, 'D': 4.253, 'E': 12.702,
    'F': 2.228, 'G': 2.015, 'H': 6.094, 'I': 6.966, 'J': 0.153,
    'K': 0.772, 'L': 4.025, 'M': 2.406, 'N': 6.749, 'O': 7.507,
    'P': 1.929, 'Q': 0.095, 'R': 5.987, 'S': 6.327, 'T': 9.056,
    'U': 2.758, 'V': 0.978, 'W': 2.360, 'X': 0.150, 'Y': 1.974,
    'Z': 0.074
}

# 常見英文雙字母組（小寫），用來衡量解密結果是否含有自然的字母組合
COMMON_BIGRAMS = {
    "th", "he", "in", "er", "an", "re", "nd", "at", "on", "nt",
    "ha", "es", "st", "en", "ed", "to", "it", "ou", "ea", "hi",
    "is", "or", "ti", "as", "te", "et", "ng", "of", "al", "de",
    "se", "le", "sa", "si", "ar", "ve", "ra", "ld", "ur",
}

# 常見英文三字母組（小寫），比 bigram 更能反映真實英文結構
COMMON_TRIGRAMS = {
    "the", "and", "ing", "ent", "ion", "her", "for", "tha", "nth",
    "int", "ere", "tio", "ter", "est", "ers", "ati", "hat", "ate",
    "all", "eth", "hes", "ver", "his", "oft", "sth", "oth", "res",
    "ont",
}

# 混合評分預設權重（當呼叫端未指定權重、也無法使用動態權重時的備援值）
CHI_WEIGHT = 0.35
BIGRAM_WEIGHT = 0.28
TRIGRAM_WEIGHT = 0.27
VOWEL_WEIGHT = 0.10
TARGET_VOWEL_RATIO = 0.38   # 一般英文文本中元音約佔 38%

# ==================== 動態權重 ====================

def get_dynamic_weights(total_letters):
    """
    根據字母總數動態調整混合評分權重。
```

```
    設計原則：
    - 短文本（< 20 字母）：卡方極不穩定，主要依賴 bigram / trigram / 元音。
    - 中文本：四項均衡。
    - 長文本（> 200 字母）：卡方穩定，提高卡方權重。

    做法：用線性插值，在「短文本權重」與「長文本權重」之間平滑過渡。

    回傳 (chi_weight, bigram_weight, trigram_weight, vowel_weight)。
    """
    # 短文本權重（偏向 n-gram 與元音）
    short_w = (0.15, 0.35, 0.25, 0.25)
    # 長文本權重（偏向卡方）
    long_w = (0.50, 0.22, 0.20, 0.08)

    # 插值參數 t：0 = 完全短文本，1 = 完全長文本
    if total_letters <= 20:
        t = 0.0
    elif total_letters >= 200:
        t = 1.0
    else:
        # 20~200 字母之間：以線性比例決定過渡程度
        t = (total_letters - 20) / (200 - 20)

    # 依 t 在 short_w 與 long_w 之間做線性插值，得到四項最終權重
    chi_w = short_w[0] + t * (long_w[0] - short_w[0])
    bg_w = short_w[1] + t * (long_w[1] - short_w[1])
    tg_w = short_w[2] + t * (long_w[2] - short_w[2])
    vw_w = short_w[3] + t * (long_w[3] - short_w[3])

    return chi_w, bg_w, tg_w, vw_w


# ==================== 字母統計與評分 ====================

def count_letters(text):
    """
    統計 text 中 A-Z 每個英文字母的出現次數。
    - 不分大小寫，全部先轉成大寫再計數。
    - 回傳的字典保證包含 A-Z 所有字母，沒出現的字母次數為 0。
    """
    # 先統一轉成大寫，再用 Counter 統計每個字元出現次數
    counts = Counter(text.upper())
    # 只取出 A-Z；未出現的字母在 Counter 中不存在，因此會得到 0
    return {char: counts[char] for char in string.ascii_uppercase}


def chi_square_score(text):
    """
    計算卡方分數（chi-square score）。
    - 分數越低，表示字母分布越接近一般英文。
    - 若文字中完全沒有英文字母，回傳無限大。
    """
    counts = count_letters(text)
    total = sum(counts.values())        # 文本中的字母總數

    if total == 0:                      # 沒有任何字母 → 無法比較分布
        return float('inf')

    chi2 = 0.0
    for letter in string.ascii_uppercase:
        observed = counts[letter]                           # 實際觀測次數
        expected = total * ENGLISH_FREQ[letter] / 100.0     # 依英文頻率推算的期望次數
        if expected > 0:                                    # 避免除以 0
            # 卡方公式：Σ (觀測 - 期望)^2 / 期望
            chi2 += (observed - expected) ** 2 / expected

    return chi2


def extract_words(text):
    """
    以非英文字母為邊界，切出所有單詞。
```

```
    - 每個單詞都是純 ASCII 英文字母組成的字串。
    - 特殊字符（空白、標點、數字等）不會被納入單詞。
    """
    # [A-Za-z]+ 會把連續的英文字母視為一個單詞，
    # 因此空白、標點、數字等天然成為單詞邊界而不被計入
    return re.findall(r'[A-Za-z]+', text)


def bigram_score(text):
    """
    計算常見雙字母組的命中率。
    - 先以特殊字符為邊界切出單詞，再對每個單詞內部算 bigram。
    - bigram 不會跨越特殊字符。
    - 回傳命中 COMMON_BIGRAMS 的比例，越高越像英文。
    """
    words = extract_words(text)

    total = 0   # 所有單詞內的 bigram 總數
    hits = 0    # 命中 COMMON_BIGRAMS 的 bigram 數量
    for word in words:
        w = word.lower()                # 轉小寫以比對小寫的常數表
        for i in range(len(w) - 1):     # 逐一切出長度 2 的連續子字串
            total += 1
            if w[i:i + 2] in COMMON_BIGRAMS:
                hits += 1

    # 若沒有任何 bigram（例如空字串或全是單字母單詞），回傳 0 避免除以 0
    return hits / total if total > 0 else 0.0


def trigram_score(text):
    """
    計算常見三字母組的命中率。
    - 先以特殊字符為邊界切出單詞，再對每個單詞內部算 trigram。
    - trigram 不會跨越特殊字符。
    - 回傳命中 COMMON_TRIGRAMS 的比例，越高越像英文。
    """
    words = extract_words(text)

    total = 0   # 所有單詞內的 trigram 總數
    hits = 0    # 命中 COMMON_TRIGRAMS 的 trigram 數量
    for word in words:
        w = word.lower()                # 轉小寫以比對小寫的常數表
        for i in range(len(w) - 2):     # 逐一切出長度 3 的連續子字串
            total += 1
            if w[i:i + 3] in COMMON_TRIGRAMS:
                hits += 1

    # 若沒有任何 trigram（單詞都太短），回傳 0 避免除以 0
    return hits / total if total > 0 else 0.0


def vowel_score(text):
    """
    計算元音比例分數。
    - 元音為 a, e, i, o, u（不分大小寫）。
    - 英文文本元音比例約 0.38，越接近越高分。
    - 回傳 0~1，1 表示比例剛好等於 TARGET_VOWEL_RATIO。
    """
    # 取出所有英文字母並轉小寫；標點、數字、其他字元一律忽略
    letters = [c.lower() for c in text
            if ('a' <= c <= 'z') or ('A' <= c <= 'Z')]

    total = len(letters)
    if total == 0:                      # 沒有字母 → 無法評估
        return 0.0

    vowels = sum(1 for c in letters if c in 'aeiou')    # 元音字母個數
    ratio = vowels / total                              # 實際元音比例

    # 距離目標比例越遠，分數越低；最大距離為 TARGET_VOWEL_RATIO
    distance = abs(ratio - TARGET_VOWEL_RATIO)
```

```
    score = max(0.0, 1.0 - distance / TARGET_VOWEL_RATIO)
    return score


def normalize_chi_score(chi2, total_letters):
    """
    把「越低越好」的卡方轉成「越高越好」的分數，範圍 (0, 1]。
    - total_letters 為 0 時回傳 0。
    """
    if total_letters == 0:
        return 0.0
    # chi2 / total_letters 是「平均每字母的卡方貢獻」，值越小越像英文；
    # 透過 1 / (1 + x) 映射到 0~1，x = 0 時分數為 1
    return 1.0 / (1.0 + chi2 / total_letters)


def compute_all_scores(text):
    """
    一次算好所有分數，避免重複計算。
    回傳 (chi2, chi_score, bg_score, tg_score, vw_score, total_letters)。
    """
    counts = count_letters(text)
    total_letters = sum(counts.values())

    if total_letters == 0:              # 完全沒有字母 → 回傳無效分數
        return float('inf'), 0.0, 0.0, 0.0, 0.0, 0

    # 依序計算四項分數（後三項本來就在 0~1，卡方需再正規化）
    chi2 = chi_square_score(text)
    chi_score = normalize_chi_score(chi2, total_letters)
    bg_score = bigram_score(text)
    tg_score = trigram_score(text)
    vw_score = vowel_score(text)

    return chi2, chi_score, bg_score, tg_score, vw_score, total_letters


def mixed_score_from_parts(chi_score, bg_score, tg_score, vw_score,
                           chi_weight=CHI_WEIGHT,
                           bigram_weight=BIGRAM_WEIGHT,
                           trigram_weight=TRIGRAM_WEIGHT,
                           vowel_weight=VOWEL_WEIGHT):
    """用已算好的分數加權，回傳混合分數，越大越好。"""
    # 四項分數皆為 0~1，權重和為 1，因此混合分數也落在 0~1
    return (
        chi_weight * chi_score +
        bigram_weight * bg_score +
        trigram_weight * tg_score +
        vowel_weight * vw_score
    )


def mixed_score(text, chi_weight=None, bigram_weight=None,
                trigram_weight=None, vowel_weight=None):
    """
    混合評分。
    - 若未指定權重，會依文本長度自動使用動態權重。
    """
    chi2, chi_score, bg_score, tg_score, vw_score, total_letters = compute_all_scores(text)

    if total_letters == 0:
        return -1.0     # 沒有字母 → 給予最低分，讓任何合法結果都能勝出

    if chi_weight is None:
        # 呼叫端未指定權重 → 依字母總數自動取得動態權重
        chi_weight, bigram_weight, trigram_weight, vowel_weight = \
            get_dynamic_weights(total_letters)

    return mixed_score_from_parts(
        chi_score, bg_score, tg_score, vw_score,
        chi_weight, bigram_weight, trigram_weight, vowel_weight
    )
```

```
# ==================== 直方圖 ====================

def print_histogram(counts, unit_symbol='█', show_zero=False):
    """
    以文字直方圖顯示字母出現的百分比。
    - 每個 unit_symbol 代表 1%。
    - show_zero 為 True 時，連次數 0 的字母也顯示。
    - 百分比 = 該字母出現次數 / 總字母數 * 100。
    """
    total = sum(counts.values())

    # 表頭與分隔線
    print(f"\nLetter Frequency Histogram ( {unit_symbol} = 1% )")
    print("-" * 60)

    if total == 0:                      # 沒有任何字母 → 只印提示訊息
        print("No letters found.")
        return

    print(f"Total letters: {total}")
    print("-" * 60)
    print(f"{'Letter':<8}{'Count':<8}{'Percent':<10}Histogram")
    print("-" * 60)

    for letter, count in counts.items():
        if show_zero or count > 0:      # 依參數決定是否顯示次數為 0 的字母
            percent = count / total * 100       # 該字母佔全部字母的百分比
            bar_len = int(round(percent))       # 1 個符號代表 1%，四捨五入
            if count > 0 and bar_len == 0:
                bar_len = 1  # 非零至少顯示一個符號
            bar = unit_symbol * bar_len
            # 依序印出：字母、次數、百分比、直方圖
            print(f"{letter:<8}({count:>3})   {percent:>6.2f}%   {bar}")


# ==================== 凱撒解密 ====================

def decryption(cipher_text, k):
    """
    凱撒解密：
    - 將每個英文字母往前移 k 位。
    - 只處理 ASCII 英文字母，大寫保持大寫，小寫保持小寫。
    - 非 ASCII 英文字母 ( 含其他 Unicode 字母、標點、數字等 ) 原樣保留。
    """
    decryption_text = []

    for char in cipher_text:
        if 'A' <= char <= 'Z':
            # 大寫字母：先轉成 0~25，減去 k 後對 26 取模，再轉回字母
            decrypted_char = chr((ord(char) - ord('A') - k) % 26 + ord('A'))
            decryption_text.append(decrypted_char)
        elif 'a' <= char <= 'z':
            # 小寫字母：同上，但以 ord('a') 為基準，維持小寫
            decrypted_char = chr((ord(char) - ord('a') - k) % 26 + ord('a'))
            decryption_text.append(decrypted_char)
        else:
            # 標點、空白、數字、非 ASCII 字元：不加密，原樣保留
            decryption_text.append(char)

    # 把字元串列接回完整字串後回傳
    return "".join(decryption_text)


# ==================== 主程式流程 ====================

def main():
    # 讀入使用者輸入的文字 ( 通常是待破解的密文 )
    user_input = input("Enter your text: ")

    # 統計字母分布，並印出頻率直方圖
```

```
    result = count_letters(user_input)
    total_letters = sum(result.values())

    print_histogram(result)

    if total_letters == 0:              # 無字母則無需嘗試解密
        print("\nNo letters found.")
        return

    possible_k_values = list(range(26))     # 凱撒位移所有可能值：0~25

    print("\nPossible decryption shifts (Caesar):")
    print(possible_k_values)

    # 依密文長度決定動態權重
    chi_w, bg_w, tg_w, vw_w = get_dynamic_weights(total_letters)
    print(
        f"\nDynamic weights (total letters = {total_letters}): "
        f"chi = {chi_w:.3f}, bigram = {bg_w:.3f}, "
        f"trigram = {tg_w:.3f}, vowel = {vw_w:.3f}"
    )

    # 記錄目前最佳的位移與分數（分數範圍 0~1，故初始值設為 -1）
    best_k = None
    best_score = -1.0

    for k in possible_k_values:
        decrypted_text = decryption(user_input, k)

        # 每個位移只需計算一次四項分數，再用同一組權重組合成混合分數
        chi2, chi_score, bg, tg, vw, _ = compute_all_scores(decrypted_text)
        score = mixed_score_from_parts(
            chi_score, bg, tg, vw,
            chi_w, bg_w, tg_w, vw_w
        )

        # 印出此位移的完整分數明細，方便檢視與除錯
        print(
            f"shift k = {k:2d} | chi2 = {chi2:8.2f} | "
            f"chi = {chi_score:.4f} | bigram = {bg:.4f} | trigram = {tg:.4f} | "
            f"vowel = {vw:.4f} | mixed = {score:.4f} | "
            f"decrypted text: {decrypted_text}"
        )

        # 分數更高就更新最佳解
        if score > best_score:
            best_score = score
            best_k = k

    if best_k is not None:
        print(f"\nBest shift by mixed score: k = {best_k} (score = {best_score:.4f})")
        print(f"decrypted text: {decryption(user_input, best_k)}")


# 只有直接執行本檔案時才啟動主流程（被當成模組 import 時不會執行）
if __name__ == "__main__":
    main()
```

13 附錄 C：各函式偽代碼

本附錄以偽代碼逐一描述附錄 B 中 14 個函式的做法，順序與原始碼相同，便於對照；最後另附介面端的主流程 `run_analysis()`，說明核心函式在圖形介面中如何被串起來使用。

閱讀記號

```
← 賦值 · 回傳 = 結束並輸出結果 · × 乘法 · mod 取餘數 · ∞ 無限大 · |…| 絕對值
對 X 中每個 Y： 逐一走訪 · 若 / 否則： 條件判斷 · 結果[i]：取出第 i 個元素
```

共用的常數與資料表

```
ENGLISH_FREQ['A' .. 'Z'] ← 26 個字母的英文出現頻率（%，總和 100）
COMMON_BIGRAMS[39] ← 常見雙字母組（th、he、in…）
COMMON_TRIGRAMS[28] ← 常見三字母組（the、and、ing…）
CHI_WEIGHT ← 0.35 BIGRAM_WEIGHT ← 0.28 TRIGRAM_WEIGHT ← 0.27 VOWEL_WEIGHT ← 0.10
TARGET_VOWEL_RATIO ← 0.38（一般英文的元音比例）
```

get_dynamic_weights(total_letters)

```
短 ← (0.15, 0.35, 0.25, 0.25) # 短文本：偏重 n-gram 與元音
長 ← (0.50, 0.22, 0.20, 0.08) # 長文本：偏重卡方
若 total_letters <= 20 ： t ← 0
若 total_letters >= 200： t ← 1
否則 ： t ← (total_letters - 20) / 180
對 i ← 0 到 3： w[i] ← 短[i] + t × (長[i] - 短[i])
回傳 (w[0], w[1], w[2], w[3])
```

count_letters(text)

```
上層 ← 把 text 轉成大寫
計數 ← 空映射
對 上層 中每個字元 c：
    若 c 屬於 'A'~'Z'： 計數[c] ← 計數[c] + 1
對 'A'~'Z' 中每個字母 L：
    結果[L] ← 計數[L]（不存在時為 0）
回傳 結果（固定 26 個鍵）
```

chi_square_score(text)

```
計數 ← count_letters(text)； n ← 26 個計數的總和
若 n = 0： 回傳 ∞ # 沒有字母，無法比較分布
chi2 ← 0
對 'A'~'Z' 中每個字母 L：
    觀測 ← 計數[L]
    期望 ← n × ENGLISH_FREQ[L] / 100
    若 期望 > 0： chi2 ← chi2 + (觀測 - 期望)² / 期望
回傳 chi2（越低越像英文）
```

extract_words(text)

```
回傳 所有符合 [A-Za-z]+ 的子字串
# 連續英文字母視為一個單詞；空白、標點、數字自然成為邊界
```

bigram_score(text)

```
單詞 ← extract_words(text)； 總數 ← 0； 命中 ← 0
對 單詞 中每個 word：
    w ← word 轉小寫
    對 i ← 0 到 len(w) - 2：
        總數 ← 總數 + 1
        若 w[i:i+2] 在 COMMON_BIGRAMS： 命中 ← 命中 + 1
若 總數 = 0： 回傳 0.0
回傳 命中 / 總數
```

trigram_score(text)

```
單詞 ← extract_words(text)； 總數 ← 0； 命中 ← 0
對 單詞 中每個 word：
    w ← word 轉小寫
    對 i ← 0 到 len(w) - 3：
        總數 ← 總數 + 1
        若 w[i:i+3] 在 COMMON_TRIGRAMS： 命中 ← 命中 + 1
若 總數 = 0： 回傳 0.0
回傳 命中 / 總數
```

vowel_score(text)

```
字母 ← text 中所有英文字母（轉小寫）； 若為空： 回傳 0.0
元音 ← 字母 中屬於 {a, e, i, o, u} 的個數
比例 ← 元音 / len(字母)
距離 ← |比例 - TARGET_VOWEL_RATIO|
回傳 max(0.0, 1.0 - 距離 / TARGET_VOWEL_RATIO)
```

normalize_chi_score(chi2, total_letters)

```
若 total_letters = 0： 回傳 0.0
回傳 1.0 / (1.0 + chi2 / total_letters) # 把「越低越好」轉成 0~1
```

compute_all_scores(text)

```
n ← count_letters(text) 的總和
若 n = 0： 回傳 (∞, 0.0, 0.0, 0.0, 0.0, 0)
chi2 ← chi_square_score(text)
chi ← normalize_chi_score(chi2, n)
bg ← bigram_score(text)
tg ← trigram_score(text)
vw ← vowel_score(text)
回傳 (chi2, chi, bg, tg, vw, n)
```

mixed_score_from_parts(chi, bg, tg, vw, w_chi, w_bg, w_tg, w_vw)

```
# 四項分數皆為 0~1，四項權重和為 1，故結果也落在 0~1
回傳 w_chi × chi + w_bg × bg + w_tg × tg + w_vw × vw
```

mixed_score(text, 四項權重可省略)

```
(chi2, chi, bg, tg, vw, n) ← compute_all_scores(text)
若 n = 0： 回傳 -1.0 # 沒有字母就給最低分
若 未指定權重： (w_chi, w_bg, w_tg, w_vw) ← get_dynamic_weights(n)
回傳 mixed_score_from_parts(chi, bg, tg, vw, w_chi, w_bg, w_tg, w_vw)
```

print_histogram(counts, unit_symbol ← '█', show_zero ← 假)

```
總數 ← counts 的總和
印出表頭（一個 █ 代表 1%）
若 總數 = 0： 印出 "No letters found."； 結束
對 counts 中每個 (字母, 次數)：
    若 show_zero 為真 或 次數 > 0：
        百分比 ← 次數 / 總數 × 100
        長度 ← round(百分比)； 若 次數 > 0 且 長度 = 0： 長度 ← 1
        印出 字母、次數、百分比與 unit_symbol × 長度
```

decryption(cipher_text, k)

```
結果 ← ""
對 cipher_text 中每個字元 c：
    若 'A' <= c <= 'Z'： 結果 ← 結果 + 位移(c, 'A', k)
    若 'a' <= c <= 'z'： 結果 ← 結果 + 位移(c, 'a', k)
    否則： 結果 ← 結果 + c # 標點、數字、非 ASCII 原樣保留
回傳 結果

其中 位移(ch, 基準, k) = chr((ord(ch) - ord(基準) - k) mod 26 + ord(基準))
```

main()

```
文字 ← 讀取使用者輸入
計數 ← count_letters(文字)； n ← 總和
print_histogram(計數)
若 n = 0： 印出 "No letters found."； 結束
(w_chi, w_bg, w_tg, w_vw) ← get_dynamic_weights(n)
最佳k ← 無； 最佳分數 ← -1.0
對 k ← 0 到 25：
    解密 ← decryption(文字, k)
    (chi2, chi, bg, tg, vw, _) ← compute_all_scores(解密)
    分數 ← mixed_score_from_parts(chi, bg, tg, vw, w_chi, w_bg, w_tg, w_vw)
    印出 k、chi2、chi、bg、tg、vw、分數 與解密文字
    若 分數 > 最佳分數： 最佳分數 ← 分數； 最佳k ← k
印出 最佳k 與 decryption(文字, 最佳k)
```

（介面端）run_analysis() —— caesar_gui.py

```
文字 ← 讀取輸入框內容
若 文字 為空： 狀態列提示「請先輸入要破解的密文」； 結束
計數 ← core.count_letters(文字)； n ← 總和
若 n = 0： 顯示「找不到任何英文字母」； 結束
權重 ← core.get_dynamic_weights(n)
對 k ← 0 到 25：
    解密 ← core.decryption(文字, k)
    (chi2, chi, bg, tg, vw, _) ← core.compute_all_scores(解密)
    分數 ← core.mixed_score_from_parts(chi, bg, tg, vw, 權重)
    rows[k] ← {k, chi2, chi, bg, tg, vw, 分數, 解密}
排名 ← rows 依分數由高到低排序
左方 ← 頻率直方圖、位移明細、候選排名、動態權重長條
右方 ← 排名第 1 名的 k、混合分數與該 k 的完整解密全文
狀態列 ← 位移數、字母總數、最可能 k、混合分數與耗時
```